

PROJECT REPORT

(Project Term June-July 2026)

Python CI/CD using Docker Agents

Submitted by

Priyanshu kumar -12407370
Somnath kumar - 12414229

Course Code: PETV137

Under the Guidance of

(Ms. Priyanka Gupta (Asst. Prof), Prof.(Dr).Harpreet Kaur)

School of Computer Science and Engineering



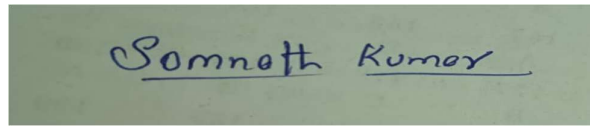
L OVELY
P ROFESSIONAL
U NIVERSITY

DECLARATION

I hereby declare that the project work entitled (“Python CI/CD using Docker Agents”) is an authentic record of my own work carried out as requirements of Project for the award of B.Tech Degree in Computer Science and Engineering from Lovely Professional University, Phagwara, under the guidance of (Name of Faculty Mentor), during June to July 2026. All the information furnished in this project report is based on my own intensive work and is genuine.

Name of Student : Somnath Kumar

Registration Number: 12414229

A rectangular box containing a handwritten signature in blue ink. The signature reads "Somnath Kumar" with a horizontal line underneath the name.

(Signature of Student)

Date: 12-07-2026

CERTIFICATE

This is to certify that the declaration statement made by the student is correct to the best of my knowledge and belief. He has completed this Project under my guidance and supervision. The present work is the result of his original investigation, effort and study. No part of the work has ever been submitted for any other degree at any University. The Project is fit for the submission and partial fulfillment of the conditions for the award of B.Tech Degree in Computer Science and Engineering from Lovely Professional University, Phagwara.

Signature and Name of the Mentor

Designation

School of Computer Science and Engineering,
Lovely Professional University,
Phagwara, Punjab.

Date :

ACKNOWLEDGEMENT

I express my sincere gratitude to the **Department of Computer Science and Engineering, Lovely Professional University**, for providing me with the opportunity to undertake this project titled "**Python CI/CD using Docker Agents**". This project has enhanced my understanding of DevOps practices, Continuous Integration/Continuous Deployment (CI/CD), Docker, Jenkins, Git, GitHub, Sonarqube, and modern software deployment techniques. I would like to sincerely thank **Prof. Dr. Harpreet Kaur and Asst. Prof. Priyanka Gupta** for their guidance, encouragement, and valuable support throughout the completion of this project. I also acknowledge the official documentation and learning resources of Docker, Jenkins, Git, GitHub, and Nginx, which helped me understand and implement the technologies used in this project. Finally, I express my heartfelt gratitude to my parents and friends for their constant encouragement, motivation, and moral support throughout the successful completion of this project.

Priyanshu Kumar , Somnath Kumar

12407370 , 12414229

B.Tech, Department of Computer Science and Engineering Lovely Professional University

Abstract

This project focuses on the automation of software delivery pipelines using Continuous Integration and Continuous Deployment (CI/CD) methodologies. The primary objective is to streamline the deployment of a Python-based microservices application (comprising a web service and a rendering service) using Docker agents. The infrastructure is distributed across three separate Amazon EC2 instances to ensure robust isolation and scalability: one instance dedicated to the Jenkins CI/CD server, another for the SonarQube static code analysis server, and a third serving as the primary Deployment server. By leveraging Docker containers, the pipeline guarantees consistency across environments. The implementation ensures that any code committed to the repository is automatically built, analyzed for code quality, and deployed seamlessly, significantly reducing manual overhead and minimizing the risk of integration issues.

1. Introduction

In modern software engineering, Continuous Integration and Continuous Deployment (CI/CD) have become essential practices to deliver high-quality applications rapidly and reliably. This project implements a fully automated CI/CD pipeline for a Python application using Jenkins as the orchestration engine and Docker as the containerization platform.

The infrastructure comprises three Amazon Elastic Compute Cloud (EC2) instances:

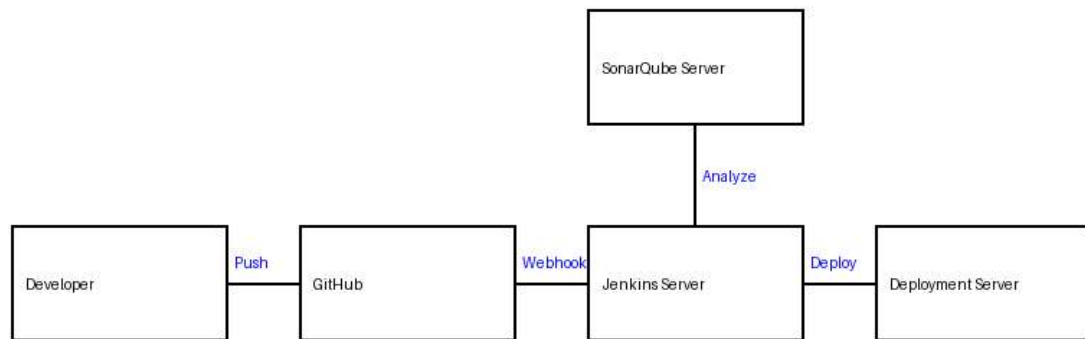
- 1. Jenkins Server: Acts as the master orchestrator for the CI/CD pipeline, coordinating builds, tests, and deployments.
- 2. SonarQube Server: Responsible for continuous inspection of code quality, detecting bugs, vulnerabilities, and code smells before deployment.
- 3. Deployment Server: The production environment where the Dockerized Python application and its dependencies (like Ollama and Node.js components) are deployed and executed.

2. System Architecture & Data Flow Diagram (DFD)

The architecture is designed to enforce a strict quality gate mechanism. Developers push their code to a remote version control repository (GitHub). This action triggers the Jenkins pipeline, which executes multiple stages inside isolated Docker containers. The source code is evaluated by the SonarQube server. Upon successful analysis, the final deployment stage uses SSH to pull the latest code and spin up the updated containers on the Deployment server.

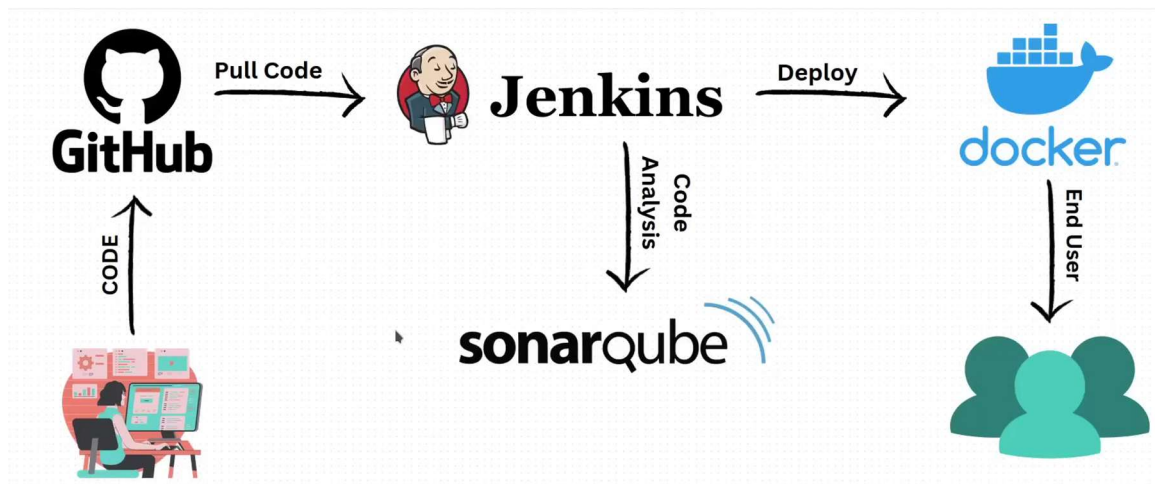
2.1 Data Flow Diagram

The following Data Flow Diagram (DFD) illustrates the flow of code and commands across the different nodes in our infrastructure:



2.2 Project Architecture

The architectural diagram represents the microservices topology deployed on the server.



3. Implementation Details

The deployment utilizes Docker Compose to manage multiple services. The primary components include the frontend web service, the Python-based rendering backend, and a local inference service (Ollama). The Jenkinsfile defines a declarative pipeline with the following key stages:

1. 1. Web Install: Uses a Node.js Docker agent to install frontend dependencies.
2. 2. Renderer Check: Uses a Python Docker agent to validate the backend code syntax and install requirements.

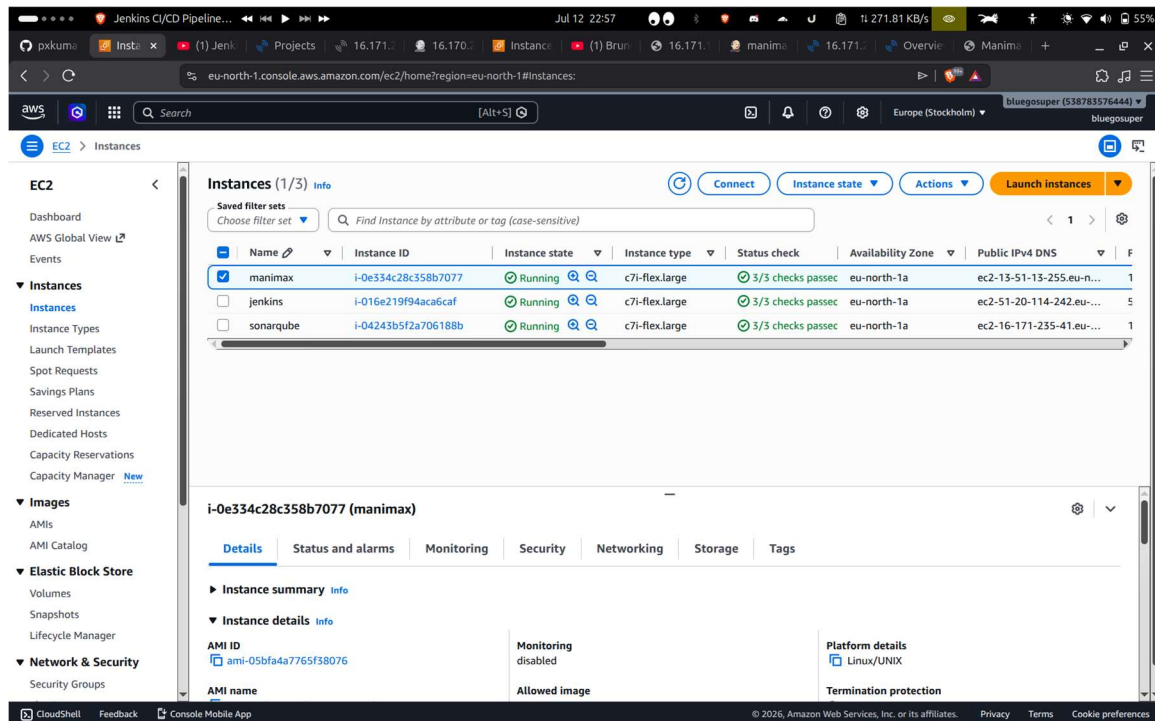
3. 3. SonarQube Analysis: Leverages the Sonar Scanner CLI container to perform static code analysis against the dedicated SonarQube EC2 instance.
4. 4. Deploy: Executes only on the main branch. It connects to the deployment EC2 instance via SSH, updates the repository, and rebuilds the Docker containers using docker-compose.

4. Testing and Results

The system was rigorously tested across multiple deployment iterations. The isolated EC2 instances successfully communicated with each other, and the pipeline correctly blocked deployments when code quality gates failed. Below are the execution results from our live servers.

4.1 AWS EC2 Instances

Three separate EC2 instances were provisioned and configured to host Jenkins, SonarQube, and the Deployment environment respectively.



4.2 Jenkins Pipeline Execution

The Jenkins pipeline executed all stages successfully. The use of ephemeral Docker agents ensured that each build step ran in a clean, reproducible environment.

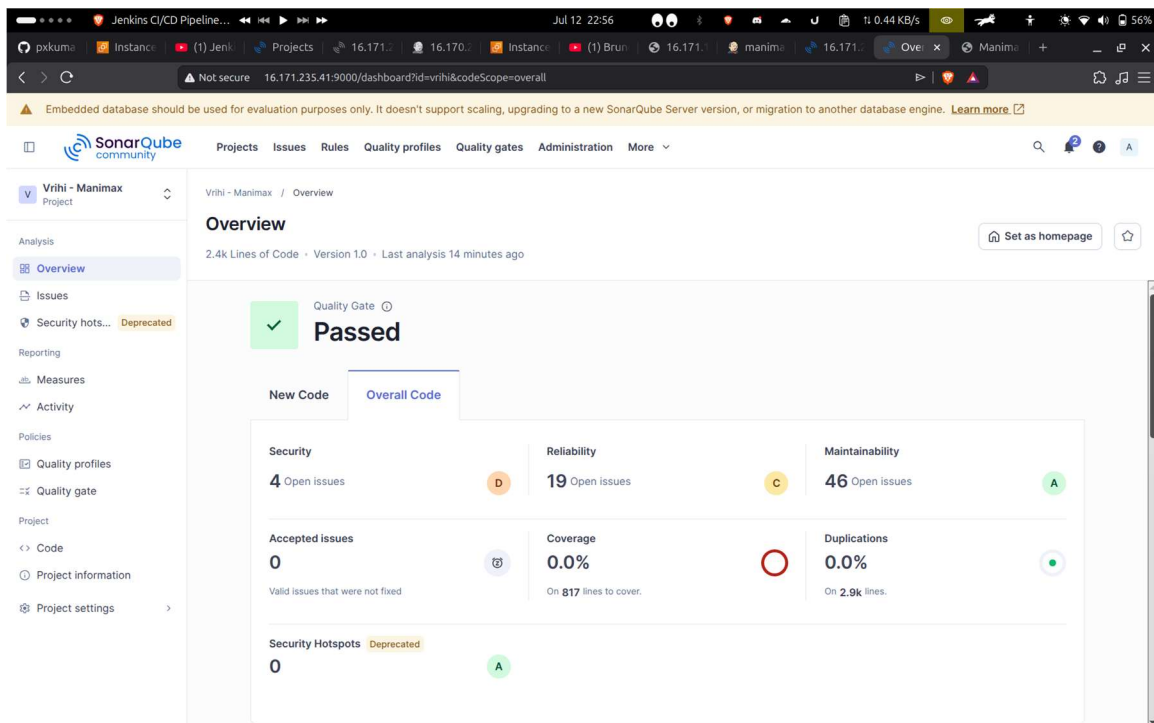
```
Jenkins CI/CD Pipeline... Jul 12 22:54
pxkuma (P) Instance d... (1) Jenkins Projects 16.171.23... 16.170.227... Instances (1) Bruno 16.171.11... manim: x Manimax: +
Not secure 51.20.114.242:8080/job/manimax/17/console

Jenkins / manimax #17

$ ssh-agent -k
unset SSH_AUTH_SOCK;
unset SSH_AGENT_PID;
echo Agent pid 11346 killed;
[ssh-agent] Stopped.
[Pipeline] // sshagent
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] }
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] node
Running on Jenkins in /var/lib/jenkins/workspace/manimax
[Pipeline] {
[Pipeline] cleanWs
[WS-CLEANUP] Deleting project workspace...
[WS-CLEANUP] Deferred wipeout is used...
[WS-CLEANUP] done
[Pipeline] }
[Pipeline] // node
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] End of Pipeline
Finished: SUCCESS
```

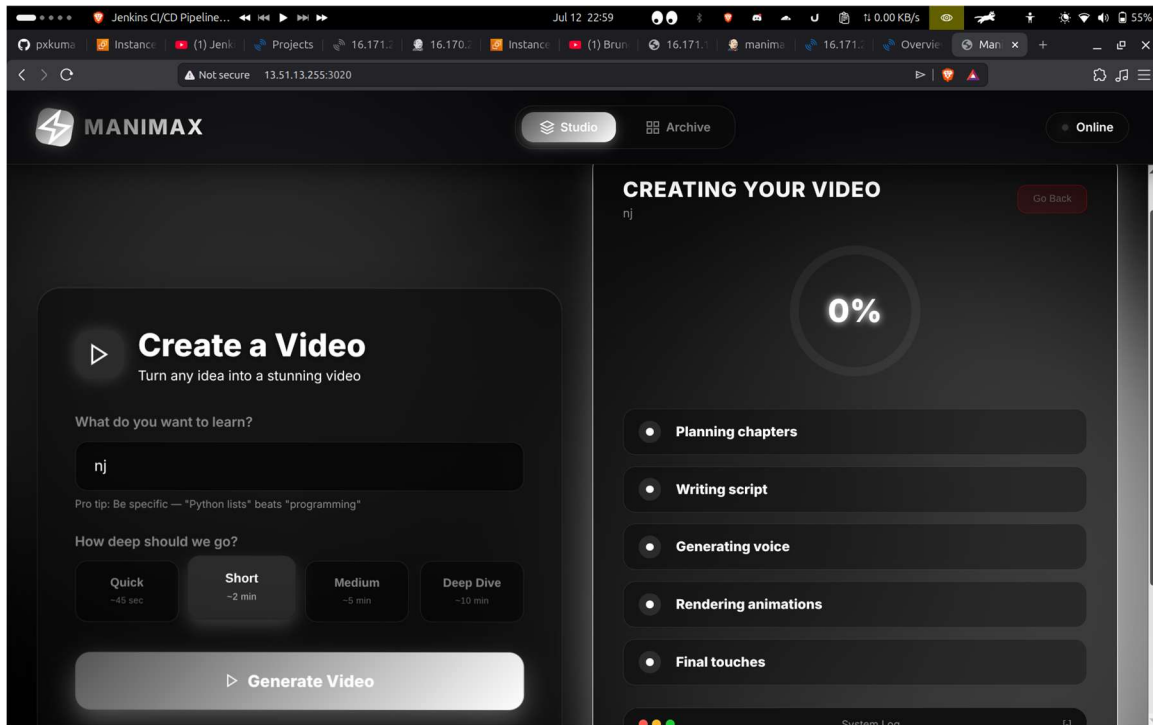
4.3 SonarQube Code Analysis

The SonarQube server processed the source code and evaluated it against predefined quality profiles. The code successfully passed the quality gate.



4.4 Application Deployment

Following the successful pipeline execution, the Python application was deployed on the production server. The containers were verified to be in a healthy, running state.



5. Conclusion

The implementation of a Docker-based CI/CD pipeline across distributed EC2 instances has proven to be highly effective. By decoupling the build, analysis, and deployment environments into three distinct servers (Jenkins, SonarQube, and Deployment), the architecture achieves superior fault tolerance and security. The integration of SonarQube guarantees high code quality standards, while Jenkins automates the delivery process, enabling rapid and reliable software releases. This project successfully demonstrates a modern, scalable approach to continuous integration and continuous deployment for Python microservices.